

Heaps Lab

Data Structures & Algorithms

Lab 07 -- Building Heaps and Putting Them to Work

Duration: 2 hours (2 challenges)

Lab Structure

Each challenge follows this format:

1. **Challenge Presentation** (5 min)
2. **Your Implementation Time** (25 min)
3. **Pseudocode Review** (5 min)
4. **Solution Walkthrough** (10 min)
5. **Code Explanation** (15 min)

Total per challenge: ~60 minutes

Today's Challenges

Challenge 1: Build a Max-Heap from Scratch

- Implement a **Max-Heap** using an array
- Support **insert**, **extractMax**, and **display** operations
- Feed it a scrambled list of exam scores and watch the highest score always sit at the top

Challenge 2: Hospital ER -- Priority Queue with a Min-Heap

- Build a **Min-Heap** based priority queue for a hospital emergency room
- Patients arrive with different severity levels (lower number = more critical)
- The most critical patient is always treated first
- Compare this with a naive unsorted-array approach by **counting operations**

Quick Recap: What is a Heap?

A **heap** is a **complete binary tree** stored in an **array** with a special ordering property:

- **Max-Heap:** Every parent is greater than or equal to its children
- **Min-Heap:** Every parent is less than or equal to its children

Array-to-Tree Mapping (0-indexed)

For a node at index i :

- **Parent:** $(i - 1) / 2$
- **Left child:** $2 * i + 1$
- **Right child:** $2 * i + 2$

Array: [90, 70, 80, 30, 50, 40, 20]
 Index: 0 1 2 3 4 5 6

Tree:

```

      90
     /  \
    70   80
   / \  / \
  30 50 40 20
  
```

$\text{parent}(5) = (5-1)/2 = 2 \rightarrow 80$
 $\text{leftChild}(1) = 2*1+1 = 3 \rightarrow 30$
 $\text{rightChild}(1) = 2*1+2 = 4 \rightarrow 50$

Challenge 1

Build a Max-Heap from Scratch

Challenge 1: Problem Statement

Scenario: You are building an exam score tracker. As students submit their exam scores, you need a system that can:

1. Accept new scores as they arrive
2. Instantly tell you the highest score in the class
3. Remove and return the highest score (e.g., to award a prize)
4. Show all scores currently in the heap

Test data (insert in this order): 40, 10, 70, 30, 90, 50, 20, 80, 60

Requirements:

- Use an array of size 100 to store the heap, and a variable `size` to track element count
- Implement `insert(value)` -- adds a value and restores the heap property using "heapify up"
- Implement `extractMax()` -- removes and returns the largest value using "heapify down"
- Implement `display()` -- prints the current heap array
- Test: insert all 9 values, display the heap, extract the top 3 scores

Time: 25 minutes

Challenge 1: Think About It First

Trace the insertions step by step. Each insert goes to the next available slot, then "bubbles up":

```
Insert 40: [40]                                Insert 10: [40, 10]

Insert 70: [40, 10, 70] --> 70 > 40, swap --> [70, 10, 40]

Insert 30: [70, 10, 40, 30] --> 30 < 10? No swap needed. Stay.
          Wait: 30's parent is at index (3-1)/2 = 1 --> value 10.
          30 > 10, swap --> [70, 30, 40, 10]

Insert 90: [70, 30, 40, 10, 90]
          90's parent = index 1 --> 30. Swap --> [70, 90, 40, 10, 30]
          90's parent = index 0 --> 70. Swap --> [90, 70, 40, 10, 30]
```

Continue inserting 50, 20, 80, 60 the same way.

Final heap array: [90, 80, 50, 70, 30, 40, 20, 10, 60] (or a valid variant)

The root (index 0) always holds the maximum!

Challenge 1: Pseudocode

ALGORITHM MaxHeap

Data: array heap[100], int size = 0

1. insert(value):

- Place value at heap[size], increment size
- HeapifyUp: $i = \text{size} - 1$
 - While $i > 0$ AND $\text{heap}[i] > \text{heap}[\text{parent}(i)]$:
 - Swap heap[i] and heap[parent(i)]
 - $i = \text{parent}(i)$

2. extractMax():

- If $\text{size} == 0$: error "heap is empty"
- Save heap[0] as maxVal
- Move heap[size - 1] to heap[0], decrement size
- HeapifyDown: $i = 0$
 - While i has at least one child:
 - Find the larger child (compare left and right)
 - If $\text{heap}[i] \geq \text{largerChild}$: STOP
 - Swap heap[i] with largerChild
 - $i = \text{index of largerChild}$
- Return maxVal

3. display(): print heap[0] to heap[size-1]

END ALGORITHM

Challenge 1: Solution Code -- Setup and Insert

```
#include <stdio.h>

#define MAX 100
int heap[MAX];
int size = 0;

void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void insert(int value) {
    if (size >= MAX) { printf("Heap full!\n"); return; }
    heap[size] = value; // Place at the end
    int i = size;
    size++;

    // Heapify Up: bubble the new value to its correct position
    while (i > 0 && heap[i] > heap[(i - 1) / 2]) {
        swap(&heap[i], &heap[(i - 1) / 2]);
        i = (i - 1) / 2; // Move to parent
    }
}
```

Challenge 1: Solution Code -- Extract Max

```
int extractMax() {
    if (size == 0) { printf("Heap is empty!\n"); return -1; }

    int maxVal = heap[0];    // The root is always the maximum
    heap[0] = heap[size - 1]; // Move last element to root
    size--;

    // Heapify Down: push the new root down to its correct position
    int i = 0;
    while (1) {
        int left  = 2 * i + 1;
        int right = 2 * i + 2;
        int largest = i;

        if (left < size && heap[left] > heap[largest])
            largest = left;
        if (right < size && heap[right] > heap[largest])
            largest = right;

        if (largest == i) break; // Heap property satisfied

        swap(&heap[i], &heap[largest]);
        i = largest;             // Continue down
    }
    return maxVal;
}
```

Challenge 1: Solution Code -- Display and Main

```
void display() {
    printf("Heap: [");
    for (int i = 0; i < size; i++) {
        printf("%d", heap[i]);
        if (i < size - 1) printf(", ");
    }
    printf("]\n");
}

int main() {
    printf("=== Challenge 1: Exam Score Tracker (Max-Heap) ===\n\n");

    int scores[] = {40, 10, 70, 30, 90, 50, 20, 80, 60};
    int n = 9;

    for (int i = 0; i < n; i++) {
        insert(scores[i]);
        printf("Inserted %d --> ", scores[i]);
        display();
    }

    printf("\nCurrent maximum score: %d\n", heap[0]);
    printf("\n--- Extracting top 3 scores ---\n");
    for (int i = 0; i < 3; i++) {
        int top = extractMax();
        printf("Extracted: %d --> ", top);
        display();
    }
    return 0;
}
```

Challenge 1: Expected Output

```
=== Challenge 1: Exam Score Tracker (Max-Heap) ===
```

```
Inserted 40 --> Heap: [40]
Inserted 10 --> Heap: [40, 10]
Inserted 70 --> Heap: [70, 10, 40]
Inserted 30 --> Heap: [70, 30, 40, 10]
Inserted 90 --> Heap: [90, 70, 40, 10, 30]
Inserted 50 --> Heap: [90, 70, 50, 10, 30, 40]
Inserted 20 --> Heap: [90, 70, 50, 10, 30, 40, 20]
Inserted 80 --> Heap: [90, 80, 50, 70, 30, 40, 20, 10]
Inserted 60 --> Heap: [90, 80, 50, 70, 30, 40, 20, 10, 60]
```

```
Current maximum score: 90
```

```
--- Extracting top 3 scores ---
```

```
Extracted: 90 --> Heap: [80, 70, 50, 60, 30, 40, 20, 10]
Extracted: 80 --> Heap: [70, 60, 50, 10, 30, 40, 20]
Extracted: 70 --> Heap: [60, 30, 50, 10, 20, 40]
```

What just happened? Every time we call `extractMax()`, the largest score pops out and the heap instantly reorganizes itself so the next-largest is at the top. No sorting needed!

Challenge 1: Code Explanation

Part 1: How Heapify Up Works (Insert)

```
while (i > 0 && heap[i] > heap[(i - 1) / 2]) {  
    swap(&heap[i], &heap[(i - 1) / 2]);  
    i = (i - 1) / 2;  
}
```

Inserting **80** into **[90, 70, 50, 10, 30, 40, 20]**:

Step 1: Place 80 at index 7 [90, 70, 50, 10, 30, 40, 20, 80]
Parent of 7 = $(7-1)/2 = 3$ --> value 10
80 > 10? YES --> swap [90, 70, 50, 80, 30, 40, 20, 10]

Step 2: Now at index 3
Parent of 3 = $(3-1)/2 = 1$ --> value 70
80 > 70? YES --> swap [90, 80, 50, 70, 30, 40, 20, 10]

Step 3: Now at index 1
Parent of 1 = $(1-1)/2 = 0$ --> value 90
80 > 90? NO --> STOP

The new element "bubbles up" the tree until it finds a parent larger than itself, or reaches the root. At most $\log n$ swaps.

Challenge 1: Code Explanation

Part 2: How Heapify Down Works (Extract Max)

```
int largest = i;
if (left < size && heap[left] > heap[largest]) largest = left;
if (right < size && heap[right] > heap[largest]) largest = right;
if (largest == i) break;
swap(&heap[i], &heap[largest]);
i = largest;
```

After extracting 90, we move the last element (60) to the root:

Start: [60, 80, 50, 70, 30, 40, 20, 10]

Step 1: i=0, children: 80(left), 50(right). Largest = 80.
60 < 80 --> swap [80, 60, 50, 70, 30, 40, 20, 10]

Step 2: i=1, children: 70(left), 30(right). Largest = 70.
60 < 70 --> swap [80, 70, 50, 60, 30, 40, 20, 10]

Step 3: i=3, children: 10(left), none(right). Largest = i.
60 > 10 --> STOP

The misplaced root value "sinks down" until both children are smaller, or it reaches a leaf. At most $\log n$ swaps.

Challenge 1: Code Explanation

Part 3: Why Use an Array Instead of Pointers?

In BSTs, we used `struct Node*` with left/right pointers. Heaps use a plain array. Why?

Feature	Pointer-based Tree	Array-based Heap
Find parent	Need a parent pointer	<code>(i-1)/2</code> -- instant!
Find left child	<code>node->left</code>	<code>2*i+1</code> -- instant!
Find last node	Must search the tree	<code>heap[size-1]</code> -- instant!
Memory overhead	2 pointers per node (16 bytes)	0 extra bytes
Cache performance	Poor (nodes scattered in memory)	Excellent (contiguous array)

The array layout works because a heap is always a complete binary tree -- there are no gaps. Every level is fully filled except possibly the last, which is filled left to right. This guarantee means the formulas `2*i+1`, `2*i+2`, and `(i-1)/2` always work correctly.

A BST cannot use an array because it may be highly unbalanced -- wasting most of the array on empty slots.

Challenge 2

Hospital ER -- Priority Queue with a Min-Heap

Challenge 2: The Scenario

You are building a **patient triage system** for a hospital emergency room. Each patient has:

- A **name**
- A **severity level** (1 = critical, 5 = minor)

The ER must always treat the **most critical patient first** (lowest severity number).

You will implement the same priority queue using TWO approaches:

1. An **unsorted array** (simple but slow)
2. A **Min-Heap** (efficient)

For each approach, count the operations during:

- Admitting 6 patients with varying severity
- Treating (removing) the 3 most critical patients

Patients (name, severity): ("Chest Pain", 1), ("Headache", 4), ("Broken Arm", 3), ("Heart Attack", 1), ("Sprain", 5), ("Breathing Issue", 2)

Time: 25 minutes

Challenge 2: Pseudocode -- Unsorted Array

```
ALGORITHM UnsortedArrayPriorityQueue

Data: array of (name, severity), int count = 0

admit(name, severity):
    - Place patient at position count (count comparisons = 0)
    - Increment count
    --> O(1) insertion, no work needed!

treatNext():
    - Scan ALL patients to find the one with the smallest severity
      (count comparisons)
    - Remove that patient by overwriting with the last patient
    - Decrement count
    --> O(n) scan every time!

END ALGORITHM
```

Insertion is fast ($O(1)$), but every time we treat a patient, we must scan the entire array to find the most critical one. With 1000 patients, that is 1000 comparisons per treatment.

Challenge 2: Pseudocode -- Min-Heap

```
ALGORITHM MinHeapPriorityQueue

Data: array of (name, severity), int count = 0

admit(name, severity):
  - Place at end, heapify up by severity (count comparisons)
  -->  $O(\log n)$  -- bubble up at most  $\log(n)$  levels

treatNext():
  - Root is always the most critical patient (severity is smallest)
  - Move last patient to root, heapify down (count comparisons)
  -->  $O(\log n)$  -- sink down at most  $\log(n)$  levels

END ALGORITHM
```

Both insert and extract are $O(\log n)$. With 1000 patients, that is about 10 comparisons -- not 1000!

Challenge 2: Solution Code -- Patient Struct and Globals

```
#include <stdio.h>
#include <string.h>

#define MAX 100

typedef struct {
    char name[30];
    int severity;    // 1 = most critical, 5 = least critical
} Patient;

// --- Unsorted Array approach ---
Patient uArr[MAX];
int uCount = 0;
int uComparisons = 0;

// --- Min-Heap approach ---
Patient hArr[MAX];
int hCount = 0;
int hComparisons = 0;

void swapPatients(Patient* a, Patient* b) {
    Patient temp = *a;
    *a = *b;
    *b = temp;
}
```

Challenge 2: Solution Code -- Unsorted Array Operations

```
// Unsorted Array: admit is O(1) -- no comparisons needed
void uAdmit(const char* name, int severity) {
    strcpy(uArr[uCount].name, name);
    uArr[uCount].severity = severity;
    uCount++;
    // No comparisons -- just append!
}

// Unsorted Array: treatNext is O(n) -- must scan all patients
Patient uTreatNext() {
    int minIdx = 0;
    for (int i = 1; i < uCount; i++) {
        uComparisons++;
        if (uArr[i].severity < uArr[minIdx].severity) {
            minIdx = i;
        }
    }
    Patient treated = uArr[minIdx];

    // Remove by overwriting with the last patient
    uArr[minIdx] = uArr[uCount - 1];
    uCount--;
    return treated;
}
```

Challenge 2: Solution Code -- Min-Heap Insert

```
// Min-Heap: admit with heapify up -- O(log n)
void hAdmit(const char* name, int severity) {
    strcpy(hArr[hCount].name, name);
    hArr[hCount].severity = severity;
    int i = hCount;
    hCount++;

    // Heapify Up: parent should have SMALLER severity (min-heap)
    while (i > 0) {
        int parent = (i - 1) / 2;
        hComparisons++;
        if (hArr[i].severity < hArr[parent].severity) {
            swapPatients(&hArr[i], &hArr[parent]);
            i = parent;
        } else {
            break;
        }
    }
}
```

Challenge 2: Solution Code -- Min-Heap Extract

```
// Min-Heap: treatNext with heapify down -- O(log n)
Patient hTreatNext() {
    Patient treated = hArr[0];          // Root = most critical
    hArr[0] = hArr[hCount - 1];        // Move last to root
    hCount--;

    // Heapify Down: find the smaller child, swap if needed
    int i = 0;
    while (1) {
        int left  = 2 * i + 1;
        int right = 2 * i + 2;
        int smallest = i;

        if (left < hCount) {
            hComparisons++;
            if (hArr[left].severity < hArr[smallest].severity)
                smallest = left;
        }
        if (right < hCount) {
            hComparisons++;
            if (hArr[right].severity < hArr[smallest].severity)
                smallest = right;
        }
        if (smallest == i) break;

        swapPatients(&hArr[i], &hArr[smallest]);
        i = smallest;
    }
    return treated;
}
```

Challenge 2: Solution Code -- Main Program

```
int main() {
    printf("=== Challenge 2: Hospital ER Priority Queue ===\n\n");
    const char* names[] = {"Chest Pain", "Headache", "Broken Arm",
                           "Heart Attack", "Sprain", "Breathing Issue"};

    int sevs[] = {1, 4, 3, 1, 5, 2};
    int n = 6;

    // Phase 1: Admit all patients
    printf("--- Phase 1: Admitting %d patients ---\n", n);
    for (int i = 0; i < n; i++) {
        uAdmit(names[i], sevs[i]);
        hAdmit(names[i], sevs[i]);
        printf("  Admitted: %-18s (severity %d)\n", names[i], sevs[i]);
    }
    printf("[Unsorted Array] Comparisons to admit: %d\n", uComparisons);
    printf("[Min-Heap]      Comparisons to admit: %d\n\n", hComparisons);

    // Phase 2: Treat the 3 most critical patients
    uComparisons = 0; hComparisons = 0;
    printf("--- Phase 2: Treating 3 most critical patients ---\n");
    for (int i = 0; i < 3; i++) {
        Patient up = uTreatNext();
        Patient hp = hTreatNext();
        printf("  Treated: %-18s (severity %d)\n", hp.name, hp.severity);
    }
    // ... (continued on next slide)
```


Challenge 2: Solution Code -- Main (continued)

```
printf("\n[Unsorted Array] Comparisons to treat 3: %d\n", uComparisons);
printf("[Min-Heap]          Comparisons to treat 3: %d\n\n", hComparisons);

// Show remaining patients
printf("--- Remaining patients in Min-Heap ---\n");
printf("  ");
for (int i = 0; i < hCount; i++) {
    printf("%s(%d) ", hArr[i].name, hArr[i].severity);
}
printf("\n");

return 0;
}
```

Challenge 2: Expected Output

```
=== Challenge 2: Hospital ER Priority Queue ===

--- Phase 1: Admitting 6 patients ---
Admitted: Chest Pain      (severity 1)
Admitted: Headache        (severity 4)
Admitted: Broken Arm      (severity 3)
Admitted: Heart Attack     (severity 1)
Admitted: Sprain           (severity 5)
Admitted: Breathing Issue  (severity 2)
[Unsorted Array] Comparisons to admit: 0
[Min-Heap]        Comparisons to admit: 4

--- Phase 2: Treating 3 most critical patients ---
Treated: Chest Pain      (severity 1)
Treated: Heart Attack     (severity 1)
Treated: Breathing Issue  (severity 2)

[Unsorted Array] Comparisons to treat 3: 14
[Min-Heap]        Comparisons to treat 3: 6
```

The unsorted array did zero work on insertion but paid the price on extraction. The Min-Heap did a little work on insertion but extracted much faster. Overall, the Heap wins!

Challenge 2: Code Explanation

Part 1: Why the Unsorted Array Struggles at Treatment Time

When we admit patients, the unsorted array just appends -- $O(1)$, zero comparisons. Sounds great!

But when treating, it must scan **every** patient to find the most critical:

```
Treat 1st: Scan 6 patients --> 5 comparisons (find min among 6)
          Remove "Chest Pain" (severity 1)
```

```
Treat 2nd: Scan 5 patients --> 4 comparisons (find min among 5)
          Remove "Heart Attack" (severity 1)
```

```
Treat 3rd: Scan 4 patients --> 3 comparisons (find min among 4)
          Remove "Breathing Issue" (severity 2)
```

```
Total: 5 + 4 + 3 = 12 comparisons (plus a few extra for edge checks)
```

For n patients and k treatments, the unsorted array does $O(k * n)$ work. If you have 10,000 patients and treat 1,000, that is 10 million comparisons!

Challenge 2: Code Explanation

Part 2: Why the Min-Heap is Efficient

The Min-Heap maintains a partial order: the root is always the smallest. So `treatNext()` grabs the root in $O(1)$, then fixes the heap in $O(\log n)$:

```
Treat 1st: Root = "Chest Pain" (severity 1). Remove root.  
          Heapify down from root: ~2 comparisons (tree height = 2)  
  
Treat 2nd: Root = "Heart Attack" (severity 1). Remove root.  
          Heapify down: ~2 comparisons  
  
Treat 3rd: Root = "Breathing Issue" (severity 2). Remove root.  
          Heapify down: ~2 comparisons  
  
Total: ~6 comparisons
```

For n patients and k treatments: $O(k * \log n)$. For 10,000 patients and 1,000 treatments: about 13,000 comparisons instead of 10 million!

A heap does not sort everything -- it only guarantees the minimum (or maximum) is at the root. This "lazy" approach is exactly why it is so efficient: it does just enough work to maintain the property you need.

Challenge 2: Code Explanation

Part 3: The Trade-off Table

Operation	Unsorted Array	Min-Heap
Admit (insert)	$O(1)$ -- just append	$O(\log n)$ -- heapify up
Treat (extract min)	$O(n)$ -- scan all	$O(\log n)$ -- heapify down
Peek at most critical	$O(n)$ -- scan all	$O(1)$ -- just read root

When does the unsorted array win? Only when you have very few extractions compared to insertions. For example, if you insert 1000 elements and only extract 1-2, the array's $O(1)$ inserts save time.

When does the heap win? Whenever you extract frequently -- which is the common case for priority queues. In a hospital ER, CPU scheduler, or network router, you extract as often as you insert.

There is also a **sorted array** option ($O(n)$ insert, $O(1)$ extract), but it does just as much total work as the unsorted array -- the cost just shifts from extraction to insertion.

Challenge 2: Code Explanation

Part 4: Min-Heap vs. Max-Heap -- Just Flip the Comparison

The only difference between Challenge 1 (Max-Heap) and Challenge 2 (Min-Heap) is the comparison direction:

```
// Max-Heap (Challenge 1): parent should be LARGER
while (i > 0 && heap[i] > heap[(i - 1) / 2]) { ... }

// Min-Heap (Challenge 2): parent should be SMALLER
while (i > 0 && hArr[i].severity < hArr[parent].severity) { ... }
```

Similarly for heapify down:

- Max-Heap: find the **larger** child, swap if parent is smaller
- Min-Heap: find the **smaller** child, swap if parent is larger

Everything else -- the formulas, the structure, the array layout -- is identical. Once you understand one, you understand both!

Lab Session Complete!

Summary: What You Built

Challenge 1: Max-Heap from Scratch

- Built a Max-Heap using an array with `insert` (heapify up) and `extractMax` (heapify down)
- Inserted 9 scores in random order -- the maximum was always at the root
- Extracted the top 3 scores one by one, and the heap reorganized itself each time

Challenge 2: Hospital ER Priority Queue

- Implemented the same priority queue using an unsorted array and a Min-Heap
- Counted comparisons to measure real work: the heap used far fewer operations overall
- Saw that the unsorted array's $O(1)$ insert comes at the cost of $O(n)$ extraction
- The Min-Heap's $O(\log n)$ for both operations makes it the best choice for priority queues

Key Concepts Reinforced

The Heap Property

- Max-Heap: parent $\geq$ children. Root = maximum.
- Min-Heap: parent $\leq$ children. Root = minimum.
- A heap is **not** fully sorted -- it only guarantees the root is the extreme value

Heapify Operations

- **Heapify Up (insert)**: new element bubbles up from bottom to its correct position
- **Heapify Down (extract)**: replacement root sinks down to its correct position
- Both are $O(\log n)$ because the tree height is $\log n$

Array Representation

- Parent: $(i-1)/2$, Left: $2*i+1$, Right: $2*i+2$
- Works because heaps are always **complete** binary trees -- no wasted slots

Complexity Summary

Operation	Max/Min Heap	Unsorted Array	Sorted Array
Insert	$O(\log n)$	$O(1)$	$O(n)$
Extract Max/Min	$O(\log n)$	$O(n)$	$O(1)$
Peek Max/Min	$O(1)$	$O(n)$	$O(1)$
Build from n elements	$O(n)$	$O(n)$	$O(n \log n)$
Search for arbitrary value	$O(n)$	$O(n)$	$O(\log n)$

The heap strikes the best balance -- both insert and extract are efficient. No other structure achieves $O(\log n)$ for both without the complexity of balanced BSTs.

Take-Home Challenge (Optional)

Extend your heap implementation with these exercises:

1. **Heap Sort:** Use your Max-Heap from Challenge 1 to sort an array. Insert all elements into the heap, then repeatedly call `extractMax()` and store the results in reverse order. Verify that the output is sorted. What is the time complexity?
2. **Build Heap in $O(n)$:** Instead of inserting elements one by one (which is $O(n \log n)$ total), implement `buildHeap()` that takes an existing array and converts it into a valid Max-Heap using the "bottom-up" approach: call `heapifyDown` on every non-leaf node starting from the last internal node down to the root. Verify it produces the same heap as repeated insertions.
3. **Think about it:** In Challenge 2, we used severity as an integer. In a real hospital, two patients might have the same severity level. How would you break ties so that patients who arrived earlier are treated first? (Hint: what extra field would you add to the Patient struct, and how would you modify the comparison?)

